Lecture 8: Memory Management

Emergency Operating Systems Study Guide

Read this first - what this lecture is really about

- Memory management is the OS job of fitting many processes into main memory safely and efficiently.
- The exam will likely test: definitions, fragmentation, allocation strategies, buddy system, address translation, paging, segmentation, and virtual memory comparison.
- Your goal: know what each technique solves, what problem it creates, and how logical addresses become physical addresses.

Study order	Topic	Why it matters
1	Requirements: relocation, protection, sharing, logical organization, physical organization	These are the reasons memory management exists.
2	Fixed vs dynamic partitioning	Core memory allocation techniques and fragmentation traps.
3	Internal vs external fragmentation + compaction	Very likely definition/comparison questions.
4	First fit, best fit, next fit	Likely algorithm/result question.
5	Buddy system	Professor can ask splitting/coalescing logic.
6	Logical, relative, physical addresses + base/bounds	Foundation for address translation.
7	Paging and segmentation	Most important technical part.
8	Summary of all memory techniques including virtual memory	High-yield comparison table.

1. One-page exam cheat sheet

Memorize these lines

• Relocation: the program may be loaded or swapped back into a different physical memory location; addresses must be translated.
• Protection: every memory reference must be checked at run time so a process cannot access memory it is not allowed to use.
• Fixed partitioning causes internal fragmentation. Dynamic partitioning causes external fragmentation.
• Compaction fixes external fragmentation by moving allocated blocks together, but it has overhead.
• First fit is usually the simplest, fastest, and often best. Best fit often performs badly because it creates many tiny holes.
• Paging divides memory into frames and processes into pages. Pages do not need to be contiguous. No external fragmentation.
• Segmentation divides a program into logical units. It matches the programmer view but still has external fragmentation.
• Paging address = page number + offset. Segmentation address = segment number + offset.
• Page table maps page number -> frame number. Segment table maps segment number -> base address + length.

Term	Fast definition
Frame	Fixed-size chunk of physical memory.
Page	Fixed-size chunk of a process.
Page table	Per-process table mapping pages to frames.
Segment	Logical program unit, such as function, procedure, stack, array, or variables.
Segment table	Per-process table holding each segment base address and length.
Internal fragmentation	Wasted space inside an allocated block/partition.
External fragmentation	Free memory exists, but it is split into non-contiguous holes.
Compaction	Moving allocated blocks so free memory becomes one contiguous block.
Overlay	Old technique where programmer splits a program so pieces are loaded as needed when full program cannot fit.

2. Why memory management is needed

- Memory is shared between the OS and user processes.
- In multiprogramming, memory must be subdivided so multiple processes can be resident at the same time.
- Ready processes need enough memory so the CPU does not sit idle waiting for swapping.
- Memory I/O is slow compared to CPU speed, so poor swapping decisions reduce CPU efficiency.
- The OS should support programs larger than available real/main memory, which leads toward virtual memory.

Exam phrase

• Memory management exists to allocate main memory efficiently, protect processes, support relocation and sharing, and keep enough ready processes in memory to maximize CPU utilization.

3. Memory management requirements

Requirement	Meaning	Exam trap
Relocation	The program can be placed in different physical locations. Logical/relative addresses must be translated to physical addresses.	Compiler/programmer usually does not know where the program will be loaded. Swapping/compaction can move it again.
Protection	A process must only access memory locations it has permission to read/write/execute.	Checks must happen at run time because the OS cannot predict every memory reference in advance.
Sharing	Multiple processes may need controlled access to the same code/data area.	Sharing must not break protection. Example: one copy of a common program/module.
Logical organization	Programs are written as modules that can be compiled, protected, and shared independently.	Segmentation matches this view because segments are logical units.
Physical organization	Memory exists in levels: main memory and secondary storage.	Moving information between these levels is an OS task.

Mental model

- Relocation asks: Where is this process now?
- Protection asks: Is this memory access allowed?
- Sharing asks: Can two processes safely use the same copy?
- Logical organization asks: Can we treat program parts as modules?
- Physical organization asks: What is in RAM and what is on disk?

4. Fixed partitioning

- Main memory is divided into static partitions at boot/system generation time.
- Partitions can be equal-size or unequal-size.
- A process can be loaded into a partition if process size <= partition size.
- If no empty partition is available, the OS may swap a process out to make room.
- Main problem: internal fragmentation, because a process usually does not fill the whole partition.
- Other problems: program may not fit and may need overlays; maximum number of active processes is limited by number of partitions.

Type	How it works	Good	Bad
Equal-size fixed partitions	Every partition has same size.	Very simple. Any process that fits can go in any partition.	Wastes memory for small programs. Large program may not fit. Fixed number of active processes.
Unequal-size fixed partitions	Partitions have different preset sizes.	Smaller jobs can use smaller partitions, reducing internal fragmentation.	Still internal fragmentation. Layout may not match real workload. Large jobs may still need overlays.

5. Placement algorithms with fixed/unequal partitions

- Equal-size partitions: placement is trivial; assign to any free partition.
- Unequal-size partitions can be handled with separate queues or a single queue.

Method	Idea	Advantage	Disadvantage
Separate queue per partition size	Each job waits for the smallest partition it fits in.	Minimizes internal fragmentation.	Large partition may stay unused while jobs wait for smaller partitions.
Single queue for all partitions	When a process arrives, choose the smallest available partition that fits.	More flexible.	Can increase internal fragmentation.

6. Dynamic partitioning

- Partitions have variable length and variable number.
- A process is allocated exactly as much memory as it needs.
- This avoids most internal fragmentation, but creates holes between allocated areas over time.
- Those holes cause external fragmentation: enough total free memory may exist, but not as one contiguous block.
- The solution is compaction: move allocated blocks together so free memory becomes one contiguous block. This costs time and overhead.

7. Internal vs external fragmentation

Fragmentation type	Where the waste is	Common cause	Fix
Internal fragmentation	Inside an allocated partition/block.	Fixed partition bigger than process; last page not fully used.	Use smaller partitions/pages; dynamic allocation; still may not disappear completely.
External fragmentation	Outside allocated blocks, in scattered holes.	Dynamic partitioning or segmentation with variable-size blocks.	Compaction, or use paging to remove the need for contiguous allocation.

50-percent rule from lecture

- Given N allocated blocks, another 0.5N blocks may be lost due to fragmentation.
- On average, about one-third of memory can become unusable because of external fragmentation.

8. Dynamic partition allocation strategies

Strategy	How it chooses a hole	Typical result
First fit	Choose the first free block large enough.	Usually simplest, fastest, and often best. May leave many allocations near front of memory.
Best fit	Choose the free block closest in size to the request.	Leaves the smallest leftover piece, but often creates many tiny unusable holes. Usually worst performer.
Next fit	Start scanning from the last placement and choose next large enough block.	Often allocates near end of memory and fragments the largest free block. Compaction may be needed more often.

High-yield example: request = 16 MB

- Best fit leaves a 2 MB fragment.
- First fit leaves a 6 MB fragment.
- Next fit leaves a 20 MB fragment.
- Do not automatically think best fit is best; the lecture says best fit is usually the worst performer because it litters memory with tiny blocks.

9. Buddy system

- Buddy system is a compromise between fixed partitioning and dynamic partitioning.
- Available block sizes are powers of two: 2^K words where L <= K <= U.
- Initially, all memory is one large block of size 2^U.
- For a request of size s, split blocks into two equal buddies until the smallest block with size >= s is created.
- When both buddies become free, they are coalesced/merged back into one larger block.
- A leaf node in the buddy tree represents a current block. If two buddy leaf nodes are both free, they should merge.

Request	Allocated buddy block	Why
100K	128K	128K is the smallest power-of-two block that can hold 100K.
240K	256K	240K cannot fit in 128K, so it needs 256K.
64K	64K	Exact match.
75K	128K	75K is greater than 64K but fits in 128K.

Buddy system trap

- Splitting creates two equal buddies.
- Merging/coalescing can only happen between true buddies of the same size, and only when both are free.
- Buddy system still may waste internal space because requests are rounded up to powers of two.

10. Address types and relocation

Address type	Meaning
Logical address	Reference to memory independent of current physical placement.
Relative address	Address expressed relative to a known point, usually the program origin.
Physical/absolute address	Actual location in main memory.

- Logical and relative addresses must be translated into physical addresses.
- Relocation is needed because a program may be loaded into different partitions during execution due to swapping or compaction.

11. Base/bounds relocation

- Base register holds the beginning physical address of the process.
- Bounds register is used to detect accesses beyond the allocated memory region. It may store length or ending address.
- Physical address = base + relative address, if the relative address is within bounds.
- Base/bounds gives both relocation and protection.
- Base and bounds are set when the process is loaded and when the process is swapped back in.

Answer pattern

- If relative address is valid: physical = base + relative.
- If relative address is outside bounds: invalid memory reference / protection fault.

12. Paging

- Physical memory is divided into equal fixed-size chunks called frames.
- A process is divided into equal fixed-size chunks called pages.
- Page size = frame size.
- A process can occupy many frames, and those frames do not need to be contiguous.
- The OS keeps a free-frame list and a page table for each process.
- Each page table entry gives the frame number containing that page.
- Paging removes external fragmentation; internal fragmentation is limited mainly to the last page of a process.
- Hardware support is needed because address translation happens during execution.

Paging item	Role
Page number	Index into the process page table.
Offset	Position inside the page/frame.
Frame number	Physical frame found from the page table.
Physical address	Frame number + same offset.

13. Paging address translation

- Logical address has n + m bits.
- Leftmost n bits = page number.
- Rightmost m bits = offset.
- Use page number to index the page table and obtain frame number k.
- Physical address = k * 2^m + offset.
- In binary, you can often just append the frame number to the offset.

Lecture example: 16-bit address, page size = 1K

- 1K = 1024 = 2^10, so the offset needs 10 bits.
- 16 total bits - 10 offset bits = 6 bits for page number.
- Maximum pages = 2^6 = 64 pages.
- Relative address 1502: page = 1, offset = 1502 - 1024 = 478.
- Page bits = 000001; offset bits = 0111011110; logical address = 0000010111011110.

14. Segmentation

- A program is divided into variable-length logical units called segments.
- Examples of segments: main program, procedure, function, local variables, global variables, stack, symbol table, arrays.
- Segmentation supports the user/programmer view of memory because programs are naturally organized into modules.
- Segments vary in length, but there is a maximum segment length.
- Address = segment number + offset.
- Paging is invisible to the programmer; segmentation is visible.
- Segmentation is similar to dynamic partitioning, but one process can occupy several non-contiguous partitions/segments.
- Segmentation has no internal fragmentation for exact segment sizes, but it can still suffer from external fragmentation.

Segment table entry	Purpose
Base / starting physical address	Where this segment begins in main memory.
Length / limit	How large the segment is; used to detect invalid offsets.

15. Segmentation address translation

- Logical address has n + m bits.
- Leftmost n bits = segment number.
- Rightmost m bits = offset.
- Use segment number to index the segment table.
- Read the segment base address and segment length.
- If offset >= segment length, the address is invalid.
- If offset < segment length, physical address = segment base + offset.

Very likely exam trap

• Paging does not check offset against page length because every page/frame has the same fixed size and the offset bits define a valid range.

• Segmentation must check offset against segment length because segments have different lengths.

16. Paging vs segmentation

Feature	Paging	Segmentation
Unit size	Fixed-size pages/frames.	Variable-size logical segments.
Programmer visibility	Invisible/transparent to programmer.	Visible; matches program modules.
Address form	Page number + offset.	Segment number + offset.
Table maps	Page -> frame.	Segment -> base address + length.
Fragmentation	No external fragmentation; small internal fragmentation in last page.	No internal fragmentation in exact segments, but external fragmentation remains.
Contiguity	Pages need not be contiguous.	Segments need not be contiguous.
Sharing/protection	Possible page-level sharing/protection.	Natural module-level sharing/protection.

17. All memory management techniques - final comparison

Technique	Core idea	Main advantage	Main problem
Fixed partitioning	Divide memory into preset partitions at boot/system generation time.	Simple.	Internal fragmentation; fixed active process limit; overlays for large programs.
Dynamic partitioning	Create variable-size partitions as programs are loaded.	Allocates exactly needed memory.	External fragmentation; needs compaction.
Simple paging	Divide memory into frames and processes into pages.	No external fragmentation; non-contiguous allocation.	Small internal fragmentation; page table overhead.
Simple segmentation	Divide program into logical variable-size segments.	Matches programmer view; easy module sharing/protection.	External fragmentation; segment table overhead.
Virtual memory paging	Paging, but not all pages must be in memory at once.	Large virtual address space; more multiprogramming.	Overhead, page faults, more complex management.
Virtual memory segmentation	Segmentation, but not all segments must be in memory at once.	Easy sharing of modules; more multiprogramming.	Overhead and external fragmentation issues.

18. Calculation templates you should know

Question type	How to solve
How many offset bits?	If page/frame size = 2^m bytes, offset bits = m. Example: 1K = 2^10, so offset = 10 bits.
How many page bits?	Page bits = total address bits - offset bits. Example: 16 - 10 = 6.
How many pages?	Number of pages = 2^(page bits). Example: 2^6 = 64 pages.
Find page number and offset from relative address	page = floor(address / page size). offset = address mod page size.
Paging physical address	Use page table to get frame k. Physical = k * page_size + offset.
Segmentation validity	Use segment table. Valid only if offset < segment length.
Segmentation physical address	Physical = segment base + offset.
Buddy allocation block size	Round request up to the next power-of-two block.

19. Likely exam questions and answers

Q: Why is memory management needed? A: To divide memory between OS and users, support multiple processes, allocate memory efficiently, swap data between disk and memory, and maximize CPU utilization.
Q: What are the five memory management requirements? A: Relocation, protection, sharing, logical organization, and physical organization.
Q: What is relocation? A: The ability to place or move a process to different physical memory locations while translating its references correctly.
Q: Why must protection be checked at run time? A: Because the OS cannot know every memory reference a program will make before execution, and the program location is unpredictable.
Q: What is internal fragmentation? A: Wasted space inside an allocated block or partition because the process/data is smaller than the allocated space.
Q: What is external fragmentation? A: Free memory is split into scattered holes; total free memory may be enough but not contiguous.
Q: What is compaction? A: Moving allocated blocks so they become contiguous and free space becomes one large block.
Q: Which allocation strategy is usually best? A: First fit is usually simplest, fastest, and often best.
Q: Why can best fit perform badly? A: It creates many very small fragments that are hard to use, so compaction may be needed often.
Q: What does the buddy system do? A: It splits memory blocks into equal power-of-two buddies until the smallest block large enough for the request is produced, then coalesces free buddies later.
Q: What are pages and frames? A: Pages are fixed-size chunks of a process; frames are fixed-size chunks of physical memory.
Q: What does a page table contain? A: For each page of a process, it contains the frame number where that page is stored.
Q: Why does paging remove external fragmentation? A: Because pages can be loaded into any free frames; they do not need contiguous memory.
Q: What is segmentation? A: A memory management scheme that divides a program into logical variable-size units such as functions, stack, arrays, and variables.
Q: Why does segmentation still suffer from external fragmentation? A: Segments are variable-size blocks, so free memory can become scattered into holes.
Q: What is the difference between paging and segmentation? A: Paging uses fixed-size invisible pages; segmentation uses variable-size visible logical program units.
Q: How does base/bounds relocation work? A: The base is added to a relative address to get a physical address, while bounds checks whether the address is within the allocated area.

20. Quick practice drills

1. A 16-bit logical address uses page size 1K. How many offset bits and page bits?
   - Answer: offset = 10 bits, page = 6 bits, maximum pages = 64.
2. Relative address 1502 with page size 1K belongs to what page and offset?
   - Answer: page 1, offset 478.
3. A process requests 240K in a buddy system. What block size should be allocated?
   - Answer: 256K.
4. Which technique has no external fragmentation but may have small internal fragmentation?
   - Answer: Paging.
5. Which technique matches program modules and supports different protection per module?
   - Answer: Segmentation.
6. A segment has length 500 and offset 700. Is the address valid?
   - Answer: No, because offset >= segment length.
7. A page table says page 3 is in frame 9, page size 1024, offset 100. Physical address?
   - Answer: 9 * 1024 + 100 = 9316.

21. Last-minute exam plan

If you have 45 minutes

- 10 min: memorize the one-page cheat sheet and requirements.
- 10 min: fixed vs dynamic partitioning, internal vs external fragmentation, compaction.
- 10 min: first fit vs best fit vs next fit + 16 MB example.
- 10 min: paging and segmentation address translation.
- 5 min: answer the practice drills without looking.

If you have 15 minutes

- Memorize: fixed = internal fragmentation; dynamic = external fragmentation; paging = no external fragmentation; segmentation = logical units but external fragmentation.
- Memorize formulas: paging physical = frame * page_size + offset; segmentation physical = base + offset after checking offset < length.
- Memorize: first fit usually best/fastest, best fit usually worst, buddy rounds up to powers of two.